

Наследяване - механизъм за създаване на нов клас, базиран на съществуващ. Новият ползва член-данните и методите на съществуващия.

Base class (^{базов} родителския) - от него се наследява

Derived class (^{наследен} произведен) - класът, който наследява от базовия

Is-a връзка - основата на наследяването

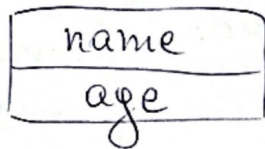
пример: Dog is-a Animal

↓ ↓
derived base

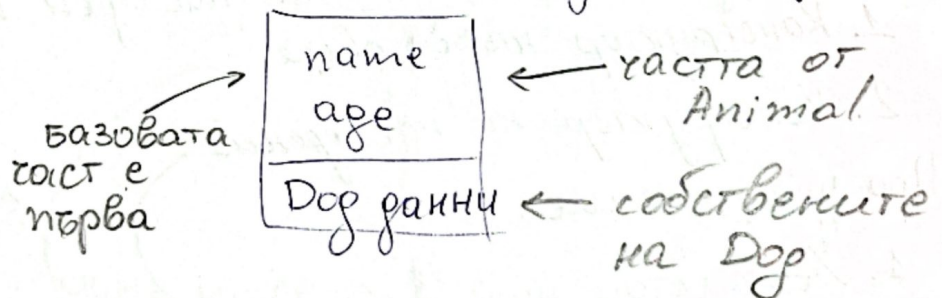
обект от наследен клас - съдържа вградените копия на базов обект + собствените си данни

1. Наследяване - когато два класа споделят обща функционалност, за да не се дублира код - единия наследява другия

Обект от тип Animal (A):



Обект от тип Dog (B наследява A):



Видове наследяване:

• Single (единично) - клас наследява от точно 1 базов клас

```
class Dog: public Animal {};
```

• Multiple (множествено) - клас наследява от 2 или повече базови класове

```
class Flyable { ... };  
class Swimmable { ... };
```

```
class Duck: public Flyable, public Swimmable  
{ ... };
```


Видимост при видовете наследяване:

член в Base ↓	public наследяване	protected наследяване	private наследяване
public	public	protected	private
protected	protected	protected	private
private	недостъпен	недостъпен	недостъпен

никога не са достъпни директно от наследения клас
⇒ пишем get-ари

Наследяване & Композиция

↓
is-а връзка
Куче е животно
промяната е трудна
(промяна в базовия
засяга всички)

↓
has-а връзка
Кола има двигател
промяната е лесна
(подменяш вътрешния обект)

Конструктори и Деструктори при наследяване

• При създаване на обект от наследен клас:

1. Конструктор на базовия
2. Конструктор на наследения

• При унищожение:

1. Деструктор на наследения
2. Деструктор на базовия

в обратен ред

В initialization list на наследения клас (конструктора) казваме кой конструктор на базовия ползваме, в противен случай - default констр.
→ задължително там!

При динамична памет всеки деструктор освобождава своята памет

Копиране:

- Ако наследения клас дефинира `copy` констр. и `op=` без да избива базовите верси, базовата част не се копира

⇒ изрично копираме базовата част

При това конструктор същото.

Ако наследения клас има собствена динамично памет, а базовият няма → Rule of Five за наследения
Rule of Zero за базовия

Подаване като параметри

<code>f1 (Base b)</code> <code>f2 (const Base &)</code> <code>f3 (const Base*)</code>	}	приемат и Base и Derived (наследен клас може да се порече навсякъде, където се иска базов)
---	---	---

<code>g1 (Der d)</code> <code>g2 (Der &)</code> <code>g3 (Der*)</code>	}	приемат само Derived
--	---	----------------------

Масив от Derived не може да се порече като масив от Base

⇒ правим масив от указатели

```
void f(const Base** arr, int size)
```

```
{ for(...)
```

```
arr[i] → method();
```

```
}
```

→ всеки указател е с
правилен размер и сочи
към обекти от масива

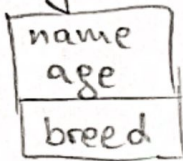
```
Derived d1, d2, d3;
```

```
const Base* arr[] = { &d1, &d2, &d3 }
```

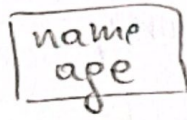
```
f(arr, 3);
```

- Slicing проблем - при подаване по стойност наследеният обект се наризва - копира само базовата част

Dog b:



Копието:

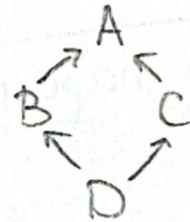


← Dog данните изглеждат

! Винаги подаване по референция или указател.

Diamond Problem

```
class A { public: int x=0; };
class B: public A {};
class C: public A {};
class D: public B, public C {};
```



D d;

d.x = 5; → двусмислено - B::A::x или C::A::x

d.B::x = 5; → изрично казваме че x

решението:

```
class B: virtual public A {};
class C: virtual public A {};
class D: public B, public C {};
```

- има само едно A в D

D d;

d.x = 5; — няма двусмислие